

CPE 334 Group Work Specification

Introduction to Software Engineering in the Age of AI Agents

Semester 1/2026 | Draft for instructor review | Issued: 6 October 2026

1. Purpose

Work in a team of two to plan, engineer, build, test, deploy, maintain, and release a full-stack software product in a new problem domain. Apply the SDD, TDD, Git, testing, and review practices learned through the individual TokTickIT labs 1-4. Use AI agents to assist with engineering work while remaining responsible for its correctness.

The project is one continuous development effort with three project submissions and three group labs performed on the same application.

Assessment	What it demonstrates
Project Deliverable 1	You can plan the product and engineer/build Sprint 1.
Project Deliverable 2	You continued the engineering discipline through Sprint 2 and into Sprint 3.
Final Delivery	You can finish, integrate, verify, and release the complete product.
Lab 5	Can you ship it? Containerize the application, establish CI/CD, and deploy it.
Lab 6	Can somebody else verify it? Obtain independent UAT and manage defects.
Lab 7	Can you change it without breaking it? Analyze and implement a change, run regression tests, and assess release readiness.

Each stage adds a new engineering responsibility. The later labs begin with your Sprint 1 release and use the evolving application as development continues.

2. Team arrangements

Unlike Labs 1–4, Labs 5, 6, and 7 are group labs. Use the same two students, repository, and application as your group project.

The purpose of a pair is to discuss decisions, challenge assumptions, review each other's work, and solve problems together. Both students must participate in requirements/design, testing, AI-agent supervision, implementation, review, deployment, and maintenance. You may distribute tasks, but you must not divide the learning outcomes into permanent roles such as “one writes specifications and the other writes code.”

Record contributions using Issues, commits, PR authorship/review, and a short contribution table. Pair work is acceptable: identify both participants in the relevant Issue or PR. Equal commit counts are not required. Both students must be able to explain a sampled requirement, test, design decision, or release step.

The default assessment is a shared team score. Suspected non-participation is referred to the instructor for review.

3. Assessment points and course weights

The course allocates **35% to labs/assignments** and **25% to the group project**. These categories have separate scoring pools.

Category / assessment	Raw points	Course Grade %
Individual Labs 1–4, existing aggregate	400	20%
Group Lab 5: CI/CD, Docker, and GCP deployment	100	5%
Group Lab 6: Cross-team UAT and defect management	100	5%
Group Lab 7: Maintenance, metrics, and release readiness	100	5%
All labs combined	700	35%
Project Deliverable 1	80	8%
Project Deliverable 2	70	7%
Final Project Delivery	100	10%
Group project subtotal	250	25%

4. Schedule

Submission deadlines are 23:59 on the given date.

Date	Activity / milestone	Points
6, 8 Oct	Miterm Exam Review Project launch: topics, scope, team workflow, engineering baseline	
12 - 25 Oct	Exam period; plan around your examination commitments	
26 Oct - 1 Nov	Midterm break; independent project work	
3, 5 Nov	Workshop: SRS, system SDS, Sprint 1 contract, tests, integration Lab 5: CI/CD, Docker, GCP deployment.	
8 Nov Sunday	Project Deliverable 1 due: baseline + working Sprint 1	80 project
10, 12 Nov	Lab 6: cross-team UAT and defect management	
15 Nov Sunday	Lab 5 due: deployed Sprint 1 and repeatable delivery	100 lab
17, 19 Nov	Lab 7: maintenance, metrics, release readiness	
22 Nov Sunday	Lab 6 due	100 lab

24, 26 Nov	Final Exam Review. Project Deliverable 2 Discussion.	
29 Nov Sunday	Deliverable 2 due Lab 7 due	70 project 100 lab
1 - 11 Dec	Final examination period	
13 Dec Sunday	Final product, report, and demo video due	100 project

By 29 November your product should be substantially complete; use the period after your examinations for final fixes, verification, and submission preparation.

5. Common product and technical requirements

5.1. Full-stack architecture

Every application must have a real **UI → API/application layer → persistent database** workflow. You may choose your stack, although you are encouraged to use the same stack as you did for Labs 1-4. Document the components and explain the choice in the System-Level SDS. A modular monolith is acceptable; microservices are not required. Three logical layers do not require three servers or three containers.

The product must support meaningful domain rules and workflows. A static site, UI mock-up, generic CRUD demonstration, or application that stores all data only in browser memory does not meet the project requirements.

Use a persistent database with documented schema, relationships, constraints, migrations/setup, and reproducible demo data. Data must survive an application restart or redeployment. Choose an architecture that can be packaged with Docker and deployed on GCP.

5.2. Authentication and authorization from Sprint 1

Every topic must include real authentication and basic server-side authorization in Sprint 1. Include:

- Login, logout, and current-user identity with a documented session/token lifecycle.
- At least two meaningful roles, as specified in the topic examples below.
- Backend enforcement of role permissions.
- Password hashing when your application manages passwords; no plaintext password storage.
- Tests for valid/invalid login, unauthenticated access, forbidden role access, and access to another user's protected data where applicable.

Seeded demo accounts are sufficient. Self-registration, password recovery, email verification, sophisticated account administration, and multiple roles per account are not required in Sprint 1. You may use suitable framework authentication libraries.

Define the role/permission matrix in your system design. Keep this foundation small enough to build alongside the domain workflow. Authentication alone is not a sufficient Sprint 1 product.

5.3. Delivery and engineering requirements

All teams must use GitHub Issues and a project board, feature/fix branches, PRs, teammate review, sprint staging/integration, automated tests, and versioned engineering documents. **Docker and GCP deployment are mandatory**, established through **Lab 5** and maintained afterward.

The delivery process must run build and automated-test checks on relevant PRs and prevent a failed required check from entering the normal release path. Deploy accepted releases from main through a repeatable pipeline; a documented manual approval within the pipeline is acceptable. A separate cloud staging environment is not required.

Document environment variables and configuration without committing real secrets. No real payment gateway, external messaging service, or external API is required - Use synthetic data.

Ways to access GCP resources will be provided in Lab 5 instructions. Design for deployment now; students are not required to have cloud resources before that guidance.

6. Ten project themes and Sprint 1 examples

We have provided 10 themes numbered 1 to 10. You are assigned to the theme whose last digit in the theme number matches the last digit of your student ID. Since you will be working with a partner, you may be fortunate to be able to choose from 2 themes. If you are unlucky to partner with someone who's got the same last digit ID, you will have a forced choice. Using your theme, specialize its context, users, and business rules. Teams with the same theme should produce independently engineered products. The descriptions of each theme provided are stakeholder briefs, not finished SRS documents. Identify ambiguities, resolve them with the instructor/TA when necessary, and record decisions.

Each example below establishes the **minimum level of Sprint 1 scope and engineering**, not a mandatory screen layout or exact feature count. An alternative vertical slice requires instructor/TA agreement and must provide equivalent depth: authentication, authorization, UI/API/database persistence, a meaningful domain transaction or transition, and a tested business rule.

1. Appointment and Scheduling System

Stakeholder brief: A service needs users to book appointments against provider availability. Staff manage available times and inspect bookings. The finished product should support availability, conflict prevention, booking, cancellation/rescheduling, schedule administration, and a useful appointment summary. Choose a context such as tutoring, advising, or consultation. Clarify slot length, cancellation limits, and who can change availability.

Sprint 1 example: A staff member logs in and creates an available slot. A customer logs in, views availability, books the slot, and retrieves their booking. Staff can view bookings for their authorized schedule. Reject booking an occupied slot and invalid/past slots; verify that competing attempts cannot create two bookings for one exclusive slot.

Sprint 1 identity/access: Customer and Staff roles. Customers see their own booking details; staff maintain only schedules they are authorized to manage. Test anonymous booking, customer access to staff operations, and another customer's private booking.

Later sprints: Cancellation/rescheduling rules, richer availability management, filtering, and appointment reporting.

2. Inventory and Borrowing System

Stakeholder brief: A department lends equipment or other assets to members. Custodians (equipment administrator) maintain the catalog and control checkout/return. The finished product should handle availability, borrowing, return, overdue status, and inventory reporting. Decide whether you track individual copies or quantities and whether requests require approval.

Sprint 1 example: Seed a small catalog. A borrower logs in, requests one available item, and sees their request. A custodian logs in and records checkout, creating an active loan and updating availability. Reject checkout of an unavailable item and duplicate active loans for the same exclusive item.

Sprint 1 identity/access: Borrower and Custodian roles. Borrowers see their own requests/loans and cannot perform checkout. Custodians manage authorized inventory and loans. Test cross-borrower access and borrower attempts to change inventory/checkout status.

Later sprints: Return, approval if applicable, due-date/overdue rules, search, and inventory summaries.

3. Event Registration and Attendance System

Stakeholder brief: Organizers publish events, manage limited places, and record attendance. Participants discover events and register. The finished product should cover publication, capacity, registration/cancellation, organizer management, check-in, and attendance reporting. Clarify duplicate registration, registration deadlines, and capacity changes.

Sprint 1 example: An organizer logs in and publishes an event with a positive capacity. A participant logs in, registers, and views their registration. The organizer views the event's registrations. Enforce capacity and one active registration per participant per event, including competing requests for the last place.

Sprint 1 identity/access: Participant and Organizer roles. Participants cannot publish events or inspect another participant's private registration. Organizers manage their own events unless broader permissions are explicitly specified. Test both role and event ownership restrictions.

Later sprints: Cancellation, registration deadlines, check-in, attendance, and summaries.

4. E-Commerce or Ordering System

Stakeholder brief: A small business accepts orders and manages fulfillment. Choose merchandise, food pre-orders, printing, or another bounded domain. The finished product should include catalog management, order creation, lifecycle transitions, simulated payment status, and order/sales summaries. Clarify price calculation, availability, cancellation, and allowed status changes. Real payments are excluded.

Sprint 1 example: Seed a small catalog. A customer logs in and submits an order with one or more line items, then opens its persisted detail. An operator logs in, views incoming orders, and accepts an order. Calculate totals on the server from authoritative prices, preserve order prices, and reject invalid quantities or unavailable items.

Sprint 1 identity/access: Customer and Operator roles. Customers see their own orders and cannot accept orders or alter authoritative prices. Operators handle orders within their specified scope. Test attempts to modify submitted totals and access another customer's order.

Later sprints: Catalog editing, cancellation/fulfillment, simulated payment transitions, and reporting.

5. Workflow and Approval System

Stakeholder brief: An organization needs controlled approval of funding, purchases, travel, reimbursement, or similar requests. Users submit requests; reviewers approve, reject, or return them with reasons. The finished product should provide role permissions, state transitions, history, and reporting. Define reviewer assignment, editing rules after submission, and any multi-stage approval.

Sprint 1 example: A requester logs in, submits a request, and views its status. An assigned reviewer logs in, reviews it, and approves or rejects it with a recorded decision/history entry. Reject an invalid transition and enforce the chosen rule preventing self-approval.

Sprint 1 identity/access: Requester and Reviewer roles. Requesters access their own requests; reviewers act only on assigned/authorized requests. Test self-approval, unrelated reviewer access, and requester attempts to use review operations.

Later sprints: Return/resubmission, additional approval stages, filtering, and decision summaries.

6. Course Project Management System

Stakeholder brief: A course or supervision program needs teams to submit milestone work and receive feedback. The finished product should manage teams, milestones, submissions, feedback, and progress. Clarify membership, deadlines, revision policy, supervision scope, and who may view feedback or grades.

Sprint 1 example: Seed teams and membership. A supervisor logs in and creates a milestone for a supervised team. A student team member logs in, submits a title/description/link, and views the persisted submission.

The supervisor records feedback. Apply the specified deadline or revision rule and preserve the submission/feedback association.

Sprint 1 identity/access: Student and Supervisor roles. Students access only their teams' submissions; supervisors access only assigned teams. Test another team's submission access and student attempts to create milestones or supervisor feedback.

Later sprints: Revision history, richer milestone planning, progress boards, and summary/report views.

7. Personal Money and Expense Logging System

Stakeholder brief: A person needs to track money, bank and wallet balances, credit card spending, personal loans, income, and expenses. The platform supports multiple users, each with private financial accounts and transactions. Users can create accounts of five types: **Cash, Loan Given, Loan Taken, Income, and Expense**. Cash accounts include wallets and bank accounts. Loan Taken accounts include credit cards and other money borrowed. Loan Given accounts represent money owed to the user. Income and Expense accounts are used to categorize transactions, such as salary, interest earned, dining, groceries, electricity, and telecom. Period budgets, shared budgets, automatic recurring transactions, and bank integration are outside scope.

The system supports two transaction forms, each with one or more allocation lines:

- **Make a payment or record a charge:** Select the cash, bank, credit card, or loan-taken account used for the transaction. Allocate the amount across expense accounts, repayments of loans taken, loans given to someone, or transfers to another cash or bank account.
- **Receive money:** Select the cash or bank account receiving the money. Allocate the amount across income accounts, repayments of loans given, new borrowing from a loan-taken account, or transfers from another cash or bank account.

Users can define opening balances and transaction dates and can view account balances, outstanding loans, credit card debt, income and expense summaries, category spending, and transaction history. Each transaction must update all relevant account balances correctly, and its allocation lines must equal its total amount. Transfers, borrowing, loan repayments, and opening balances must not be counted as income or expenses. Users can mark an existing transaction as a favorite, view favorite transactions, and select **Duplicate** to create a new editable transaction with the same details.

Sprint 1 example: Seed a few users with a bank account, a credit card account, and several income and expense accounts. A user records an opening bank balance, records salary received into the bank account, records a credit card purchase with separate dining and grocery allocation lines, and records repayment of the credit card from the bank account. The system shows the resulting bank balance, outstanding credit card debt, income and expense summary, and transaction history.

Sprint 1 identity/access: User and Administrator roles. Users manage only their own financial accounts, transactions, favorites, and summaries. Administrators manage platform user accounts and access, including enabling or disabling users, but cannot view or modify users' financial records or summaries. Enforce these restrictions on the backend. Test unauthenticated access, access to another user's records, and administrator attempts to access financial data.

Later sprints: Create and manage cash, bank, loan-taken, loan-given, income, and expense accounts. Support loans given and taken, loan repayments, transfers, opening-balance management, transaction corrections, transaction search and filtering, favorite and duplicate transactions, category spending summaries, and reports.

8. Habit and Activity Tracking System

Stakeholder brief: Users define activity goals, record progress, and review history. Choose study habits, exercise, sleep, or another routine. The finished product should handle goals, valid daily entries, progress calculations, history, and reminder/schedule representation. Specify dates, units, duplicate-entry behavior, and goal changes. This is a recording system; medical diagnosis and treatment advice are outside scope.

Sprint 1 example: A program manager logs in and defines an activity template with a unit and valid range. A participant logs in, selects a template, sets a goal, records daily activity, and sees computed progress. Enforce the specified one-entry-per-day or accumulation rule and valid values/dates.

Sprint 1 identity/access: Participant and Program Manager roles. Managers maintain templates; participant histories remain private unless the SRS explicitly defines consent-based sharing. Managers do not automatically gain access to private activity records. Test cross-participant access and participant attempts to edit shared templates.

Later sprints: Goal revisions, streak/history logic, reminder placeholders, and progress summaries.

9. Queue and Service Management System

Stakeholder brief: A service counter or restaurant needs to manage a waiting queue. The finished product should support joining, calling, serving, completing, cancelling, and handling no-shows with useful service summaries. Define queue ordering, service scopes, duplicate entries, and transition permissions. Its central workflow is a waiting line; advance appointment booking is outside the core scope.

Sprint 1 example: Seed a service queue. A customer logs in, joins it, and sees their queue number/status. An operator logs in, calls the next eligible entry, and completes service. Generate unique numbers safely, prevent duplicate active entries, and reject out-of-order or invalid transitions according to the specified queue rule.

Sprint 1 identity/access: Customer and Operator roles. Customers access their own detailed queue entry; operators act only on authorized queues. Public queue displays may show limited non-private status. Test customer attempts to call/complete entries and operator access to unauthorized queues.

Later sprints: Cancellation/no-show rules, multiple queues/counters, priority policy if applicable, and summaries.

10. Review, Feedback, and Survey System

Stakeholder brief: An organization creates surveys, collects responses, and reports aggregate results. The finished product should define surveys, control response eligibility, support identified/anonymous modes, and provide analysis/export. Clarify question types, publication/closing, duplicate responses, and access to raw data. Anonymous mode requires a documented privacy design.

Sprint 1 example: A survey owner logs in, creates and publishes a small identified survey. An eligible respondent logs in, submits a validated response, and sees confirmation. The owner views an aggregate summary. Reject repeat submissions and responses to unpublished/closed surveys.

Sprint 1 identity/access: Respondent and Survey Owner roles. Owners manage their own surveys; respondents cannot edit surveys or inspect another respondent's answers. Test unauthorized survey management and cross-response access. Anonymous response mode may be introduced later with identity/answer separation specified and tested; do not label identifiable responses anonymous.

Later sprints: Anonymous mode, more question types, closing/eligibility controls, and richer analysis/export.

7. The three-sprint development model

Sprint 1: Small deployable product

Deliver a working increment by **8 November**. Include the authentication/authorization foundation and an end-to-end domain slice comparable to Section 6. Demonstrate a normal workflow, a meaningful business-rule rejection, and access restrictions with tests.

The release must run locally with documented setup and be suitable for containerization. **GCP deployment and the completed Docker/CI/CD setup are due in Lab 5, not Deliverable 1.** Continue using this increment as the starting point for delivery, UAT, and maintenance labs.

Sprint 2: Core expansion

Implement the major remaining product workflows under feature engineering contracts. Produce an integrated, tested working increment. Sprint 2 must be complete by Deliverable 2 on **29 November**.

Sprint 3: Completion and hardening

Complete remaining planned features, integration, failure handling, usability, authorization, regression tests, and defect correction. By 29 November, provide Sprint 3 contracts, working progress, and an explicit remaining-work list. Complete and release Sprint 3 by **13 December**.

Plan the whole product's scope, architecture, and three-sprint allocation before Sprint 1 implementation. Elaborate each feature contract before implementing that feature; you do not need every Sprint 3 screen/API detail fully frozen on launch day. Record justified changes instead of silently shrinking scope or rewriting history.

8. Engineering workflow and artifacts

8.1. Product baseline

Maintain a concise **SRS** with uniquely identified Functional Requirements (**FR**), Business Rules (**BR**), and measurable important Non-Functional Requirements (NFR). Record actors, scope/exclusions, assumptions, and decisions. Build a **Feature Inventory** and **requirements-to-feature-to-sprint traceability matrix**.

Your System-Level **SDS** defines architecture/stack, major data model, UI structure/conventions, API approach, roles/permissions, configuration, and planned deployment. Use diagrams where they clarify a decision; no fixed quota of UML diagrams is required.

8.2. Feature engineering contract

For each feature, prepare before implementation:

- Feature-Level SDS: behavior, workflow, business rules, validation, errors, permissions, and dependencies.
- UI/API/data specifications where applicable: screen behavior, endpoint request/response/error shapes, schema/constraints/migrations.
- Acceptance Criteria (AC) with stable IDs and observable outcomes.
- Software Test Specification (STS): test IDs, inputs/preconditions, expected results, level, and AC coverage.
- Definition of Done and evidence required to demonstrate completion.

Link each AC to planned and actual tests. Use

- unit tests for important logic
- API/database integration tests for persistence and permissions
- UI/end-to-end tests for selected critical journeys.

Include negative/boundary tests. A stub assertion, mock-only persistence demonstration, screenshot, or AI statement that “all tests pass” is not execution evidence.

Use TDD for meaningful rule/defect/change work: show a failing test for the intended behavior, then implementation and passing tests. Do not weaken tests to make incorrect behavior pass. Document justified manual checks, but do not substitute them for the required automated coverage.

8.3. Git and release workflow

Use the course workflow:

1. Create an Issue linked to the requirement/feature and contract.
2. Create a feature/fix branch from the current sprint staging branch.
3. Implement the bounded change, run tests, inspect the diff, and update documents.
4. Open a PR into sprint staging; the teammate reviews behavior, tests, and changes.
5. Integrate features and run sprint integration/system tests and acceptance checks.
6. Open a reviewed release PR from sprint staging to main when checks pass.
7. Tag/record the accepted release and, after Lab 5, deploy it through the delivery pipeline.

Failed integration returns to an Issue/fix branch and re-testing. Reviews must contain a specific check, question, finding, or reason for approval. Authors do not approve their own PRs. Preserve contract revisions and implementation history so the sequence can be verified. **CI becomes mandatory in Lab 5; before that, retain local test results tied to the submitted commit.**

Cross-team UAT begins in Lab 6 and re-tests the relevant released version after fixes. Lab changes may be part of a planned sprint, but identify their assessment evidence clearly and keep it separate from unrelated feature work.

8.4. AI use

You may use specification and coding agents. Supply the relevant engineering contract, work on bounded Issues, inspect output, run tests, and review changes before accepting them. Both students must understand the result. Keep a short AI-use declaration listing tools and tasks assisted; full transcripts and long reflection essays are not required.

9. Project deliverables and rubrics

Detailed assessment instructions will use these categories and points. The master specification establishes the required scope. Submit a concise evidence index rather than reproducing the repository as a report.

Project Deliverable 1: Engineering baseline + working Sprint 1

Due 8 November | 80 project points | Proves you can plan the product and engineer/build Sprint 1.

Part	Points	Required evidence
SRS and product scope	10	Actors, FR/BR/NFR, scope, exclusions, assumptions, resolved decisions.
Feature inventory and traceability	10	Requirement → feature → planned sprint; coverage of the baseline.

System-Level SDS	15	Architecture/stack, data model, UI/API structure, role matrix, configuration/deployment plan.
Three-sprint plan	5	Goals, feature allocation, dependencies, feasible Sprint 1 and remaining work.
Sprint 1 engineering contracts	20	Feature SDS, applicable UI/API/data specs, AC, STS, DoD, pre-implementation history.
Working Sprint 1 and verification	20	Domain journey, persistence, authentication/access tests, business-rule tests, Issue/PR/review/integration evidence.
Total	80	

Provide a runnable release on main, its tag/commit, setup instructions, test command/results, and a brief reproducible demo sequence. Cloud deployment is not required yet.

Project Deliverable 2: Sprint 2 completion + Sprint 3 progress

Due 29 November | 70 project points | Proves you continued the discipline of Sprint 2 → Sprint 3.

Part	Points	Required evidence
Sprint 1 continuity and change status	5	Released baseline, change/defect status, resolved and remaining work.
Sprint 2 contracts and working increment	25	Pre-implementation contracts, AC/tests, working new workflows, reviewed Issues/PRs, increment release.
Sprint 3 contracts and working progress	15	Specified remaining features, tests, demonstrated implemented progress, explicit remaining-work list.
Updated requirements/test traceability	10	FR/BR/NFR → feature → sprint → AC/verification → actual test/result, with honest completion status.
Integrated release candidate	15	Sprint 1 + 2 + implemented Sprint 3, CI/deployment tied to commit, regression evidence, exam-period completion plan.
Total	70	

Submit a deployed integrated release candidate. All Sprint 2 features must be complete. Sprint 3 may have bounded unfinished work, clearly identified; placeholders do not count as implemented progress.

Final Project Delivery: Complete verified release

Due 13 Dec. | 100 project points | Proves you can integrate, verify, and release the complete product.

Part	Points	Required evidence
Final working product	35	Completed major workflows and domain rules across all three sprints; failure handling, persistence, permissions.
Engineering process and team evidence	15	Traceable Issues/PRs/reviews/integration/releases and both students' participation.

Part	Points	Required evidence
Final requirements and verification traceability	20	Complete baseline coverage; requirement → feature → AC/verification → test → final result.
Final QA and Product Definition of Done	20	Passing automated/regression tests, UAT/re-tests, critical defect status, deployed release, reproducible setup, known debt.
Demonstration video	8	Maximum 10 minutes; both students participate; live product workflows, a rejected case, brief architecture, test/CI and traceability example.
Final release summary	2	Maximum one page: scope, version/commit, links, completion, limitations, team.
Total	100	

Submit **one concise final PDF report, repository/release reference, deployed application URL, and demonstration video**. The PDF serves as an evidence index and includes the release summary. Show the running product in the video, with one important business rule, one failure/access case, and one requirement-to-test chain.

10. Group labs using the Sprint 1 release

Lab 5: CI/CD, Docker, and GCP deployment

Due 15 November | 100 lab points | Can you ship it?

Containerize the accepted Sprint 1 release, establish CI/CD, and deploy it on GCP. Demonstrate a successful release and a controlled failing check that blocks the normal merge/release path. Use a temporary branch for the failure demonstration; do not deliberately break the released application.

Part	Points	Evidence
CI and quality gate	30	PR build/tests, meaningful test results, failed required check, enforced gate.
Docker and reproducible configuration	20	Build/run configuration, fresh build, working container, externalized configuration.
GCP deployment and persistence	35	URL, deployed domain workflow, UI/API/DB connectivity, data survives restart/redeployment.
Repeatable release pipeline	15	Reviewed change → CI → merge → deployment; release commit identification and repeatable procedure.
Total	100	

Lab 6: Cross-team UAT and defect management

Due 22 November | 100 lab points | Can somebody else verify it?

Find another project team to pair with. Each team prepares UAT cases for its own released application, tests the partner's application, and triages/fixes findings on its own application. Each team is graded on both its tester and developer responsibilities. Test a recorded release, not a changing unidentified version.

Provide at least four user-facing cases: a normal journey, a domain-rule/boundary case, a role/ownership restriction, and a state/history/persistence check. Record UAT ID, user goal, preconditions, steps, expected/actual results, PASS/FAIL, tester, version, and defect link when applicable.

Part	Points	Evidence
UAT preparation and handoff	20	Executable cases, safe accounts/data, release/version, access handoff.
Independent execution	25	Partner application results, identified testers/version, evidence for findings.
Defect reporting and triage	20	Reproduction, expected/actual, severity; accepted/duplicate/not-defect/enhancement with rationale.
Resolution and independent re-test	35	Accepted defect → regression test → fix → reviewed PR/CI/deployment → partner re-test.
Total	100	

Resolve and independently re-test at least one accepted defect. If no legitimate defect is found, the TA provides a controlled defect exercise with a known failing behavior on an isolated version/branch. Do not fabricate findings or substitute a new feature for a defect fix. Partner delays or access problems should be reported promptly for reassignment/assistance.

Lab 7: Maintenance, metrics, and release readiness

Due 29 November | 100 lab points | Can you change it without breaking it?

Apply one instructor/TA-approved change to an already implemented behavior or business rule. A feature already in the baseline backlog does not by itself constitute this maintenance exercise. Trace the impact, update the contract, demonstrate the changed test failing before implementation, implement the change, run regression tests, review, and redeploy.

Part	Points	Evidence
Change impact analysis and updated contract	25	Change request → affected requirement/feature/SDS/AC/tests/code; reviewed contract revision.
Implementation and regression-safe delivery	50	Failing changed-behavior test, implementation, passing/regression results, review/CI/deployment.
Objective engineering metrics	15	Five dated metrics with sources and one action-oriented interpretation, maximum 100 words.
Release readiness and short retrospective	10	Defect/test/debt/configuration/doc/release checks plus three short retrospective answers.

Total	100
--------------	------------

Use metrics such as completed Issues, merged PRs, passing tests, UAT defects opened/resolved, and successful/failed CI runs. Counts do not earn points for being large. State the time window and source. Retrospective: what helped, what hurt, and what will change, one or two sentences each.

11. Submission and evidence conventions

Use LEB2 with one submission per team for each assessment. Identify the team, both students, assessment, repository, submitted tag/commit, deployed version where applicable, and evidence links. Maintain documents in the repository; suggested locations are docs/srs.md, docs/features.md, docs/traceability.md, docs/system-sds.md, docs/sprints/, and docs/labs/. Equivalent clear organization is acceptable.

Every assessment must have a short evidence index:

Rubric part	Artifact / direct link	IDs / selected version	Verification result or status
Example: Sprint contract	Repository document	F-03, AC-07, TEST-12, submitted commit	Planned then implemented; test passing

For tests, identify the file/test name, command or CI run, result, and tested commit. Screenshots supplement direct evidence. Keep links accessible to graders and partner testers. Keep the final deployment available through grading; the instructor will announce the end of the access period.

TAs will sample requirements, AC/tests, Issues/PRs, and deployed journeys. You must maintain complete evidence even though not every row will be inspected. Marks reward observable behavior, traceability, meaningful tests, and credible execution evidence. Document length, prompt count, commit count, and visual polish alone do not establish engineering quality.

12. Companion assessment documents

Detailed student handouts for each Project deliverable 1–3 and Lab 5–7 will be issued separately.